

Content Digest Tool – Project Plan

Context

A daily content aggregation tool that fetches emails from Gmail, summarizes each with Claude, ranks the results using a Claude + heuristic blend, and outputs a digest to stdout and a local markdown file. v0 is a command-line Ruby pipeline following existing conventions in the bin/ scripts project.

Decisions Made

Decision	Choice	Rationale
Language	Ruby	Matches existing script conventions
Email source (v0)	Gmail API (OAuth2)	User specified Gmail for v0
LLM for summarization	Claude API (claude-sonnet-4-6)	User choice; matches Anthropic toolchain
Ranking	Claude score (0-10) + sender priority + recency heuristic	User requested combination approach
Storage format	Local markdown files	Explicit user requirement for v0
Script structure	3 phase scripts + orchestrator	Matches bin/job_search_code/ multi-script pattern
Output (v0)	stdout + local .md file	User specified; no web interface yet
Digest delivery (v0)	No email send	Deferred to v1
External gems	google-apis-gmail_v1 only	Required for Gmail API; no alternative with stdlib

Deferred Decisions

Item	Status	Notes
Email source: IMAP	Deferred to v1	User confirmed all three sources eventually
Email source: MBOX	Deferred to v1	Existing mbox_ops/ code can be adapted

Item	Status	Notes
Digest delivery via email/SMTP	Deferred to v1	v0 is stdout + markdown only
Web interface for summaries	TBD	“Location and interface serving them is TBD”
Schedule mechanism	Deferred	Cron assumed; specifics TBD
Deduplication across runs	Deferred	v0 treats each daily run independently
Sender filter configuration	TBD	Currently hardcoded constant; may move to config file
Summary storage for linking	TBD	Links in digest are local paths; web server interface TBD

Trade-offs: v0 vs Later Versions

Gem dependency (Google APIs)

- **v0:** Requires `google-apis-gmail_v1` gem, departing from stdlib-only pattern in other scripts
- **Later:** If IMAP is added, Gmail can be accessed via `net/imap` (stdlib) with app password, eliminating the gem

Single API key per script vs shared config

- **v0:** Constants defined per-file (`ANTHROPIC_API_KEY`, `SENDER_FILTER`)
- **Later:** Externalize to a YAML config file shared across scripts

Hardcoded sender filter

- **v0:** `SENDER_FILTER` array constant in `fetch_gmail.rb`
- **Later:** Config file or CLI flag; supports adding/removing sources without code changes

Ranking heuristic

- **v0:** Simple weighted formula (`claude_score * 0.7 + sender_priority * 0.2 + recency * 0.1`)
- **Later:** Tunable weights; learning from user feedback; topic clustering

Script orchestration via `system()`

- **v0:** `digest.rb` calls phase scripts via `system(RbConfig.ruby, script)`

- **Later:** Rake tasks or a lightweight job runner; retry logic per phase

Output format

- **v0:** Markdown file with local relative links to summaries
- **Later:** HTML digest; email delivery; web-served summary pages

Architecture

```
~/Desktop/aggregation/
digest.rb           # orchestrator
fetch_gmail.rb      # Phase 1: Gmail API fetch -> data/raw/
summarize.rb        # Phase 2: Claude summarization -> data/summaries/
build_digest.rb     # Phase 3: Rank + digest -> data/digests/ + stdout
Gemfile
credentials.json    # TBD -- from Google Cloud Console (not committed)
token.yaml          # TBD -- auto-generated on first OAuth run (not committed)
prompts/
  summarize.txt     # Claude prompt: structured summary + score
  rank.txt          # Claude prompt: editorial narrative
data/
  raw/              # YYYYMMDD_subject.md
  summaries/        # YYYYMMDD_subject_summary.md
  digests/          # YYYYMMDD_digest.md
```

Configuration (per-file constants – v0)

Constant	File	Value	Status
SENDER_FILTER	fetch_gmail.rb	Array of email addresses	TBD – populate before first run
LOOKBACK_DAYS	fetch_gmail.rb	1	Set
MAX_RESULTS	fetch_gmail.rb	50	Set
CLAUDE_MODEL	summarize.rb, build_digest.rb	claude-sonnet-4-6	Set
ANTHROPIC_API_KEY	summarize.rb, build_digest.rb	ENV.fetch('ANTHROPIC_API_KEY', nil)	Required API key
SLEEP_BETWEEN_CALLS	summarize.rb	2 seconds	Set
TOP_N	build_digest.rb	5	Set
SENDER_PRIORITY	build_digest.rb	Hash of email -> priority (1-3)	TBD – populate before first run

Constant	File	Value	Status
<code>credentials.json</code>	<code>fetch_gmail.rb</code>	OAuth2 client credentials	TBD – obtain from Google Cloud

Phase Details

Phase 1 – `fetch_gmail.rb`

1. Authorize via OAuth2 (first run opens browser; token cached in `token.yaml`)
2. Build Gmail search query: `from:(addr1 OR addr2) after:YYYY/MM/DD`
3. Fetch matching messages (up to `MAX_RESULTS`)
4. Extract: sender, subject, date, plain-text body
5. Write each to `data/raw/YYYYMMDD_sanitized_subject.md` with YAML header block

Phase 2 – `summarize.rb`

1. Read all `.md` files from `data/raw/`
2. POST each to Claude API with `prompts/summarize.txt`
3. Parse importance score from Claude's response
4. Write to `data/summaries/YYYYMMDD_subject_summary.md` with updated header
5. Sleep between calls for rate limiting

Phase 3 – `build_digest.rb`

1. Read all `_summary.md` files
2. Compute weighted score per summary
3. Sort descending; take top `TOP_N`
4. POST ranked summaries to Claude with `prompts/rank.txt` for editorial note
5. Build digest markdown with links to full summaries
6. Print to stdout; save to `data/digests/YYYYMMDD_digest.md`

Ranking Formula

$$\text{weighted_score} = (\text{claude_score} * 0.7) + (\text{sender_priority} * 0.2) + (\text{recency} * 0.1)$$

- `claude_score`: 1-10 parsed from Claude's summary response
- `sender_priority`: 1-3 from `SENDER_PRIORITY` hash (default 1 if unlisted)
- `recency`: 1.0 for today, -0.1 per day older, minimum 0.5

Next Steps

1. **Obtain Google OAuth2 credentials** (immediate blocker for Phase 1)
 - Create a Google Cloud project (or use existing)
 - Enable Gmail API
 - Create OAuth2 credentials (Desktop app type)
 - Download `credentials.json` to `~/Desktop/aggregation/`
2. **Set environment variable**
 - `export ANTHROPIC_API_KEY=your_key_here` in shell profile
3. **Install gems**
 - `cd ~/Desktop/aggregation && bundle install`
4. **Configure sender lists**
 - Populate `SENDER_FILTER` in `fetch_gmail.rb`
 - Populate `SENDER_PRIORITY` in `build_digest.rb`
5. **First run / auth flow**
 - Run `ruby fetch_gmail.rb` – browser prompt will appear for OAuth
 - Confirm emails appear in `data/raw/`
6. **Test summarization**
 - Run `ruby summarize.rb` – confirm `data/summaries/` populates
7. **Test digest**
 - Run `ruby build_digest.rb` – confirm output to `stdout` and `data/digests/`
8. **End-to-end test**
 - Run `ruby digest.rb` – confirm all phases complete
9. **Set up cron** (after v0 is stable)
 - `0 7 * * * cd ~/Desktop/aggregation && ruby digest.rb >> /tmp/digest.log 2>&1`

Accomplished (v0 Scaffold)

- ☒ Project plan written
- ☒ Directory structure created
- ☒ `fetch_gmail.rb` – Phase 1 script (pending `credentials.json`)
- ☒ `summarize.rb` – Phase 2 script (pending `ANTHROPIC_API_KEY`)
- ☒ `build_digest.rb` – Phase 3 script (pending `ANTHROPIC_API_KEY`)
- ☒ `digest.rb` – Orchestrator script
- ☒ `prompts/summarize.txt` – Claude summarization prompt
- ☒ `prompts/rank.txt` – Claude editorial prompt
- ☒ Gemfile

Missing / TBD

- ☐ `credentials.json` – obtain from Google Cloud Console

- ☐ `token.yaml` – auto-generated on first OAuth run
- ☐ `SENDER_FILTER` values – user to populate
- ☐ `SENDER_PRIORITY` values – user to populate
- ☐ `ANTHROPIC_API_KEY` – set in shell environment
- ☐ Gem install (`bundle install`)
- ☐ First OAuth authorization run
- ☐ Cron schedule configuration
- ☐ Web interface for summary browsing (v1+)
- ☐ IMAP / MBOX source adapters (v1+)
- ☐ Email delivery of digest (v1+)

Here's the status summary:

Accomplished * `PLAN.md` – full project plan with decisions, trade-offs, deferred items, next steps * `fetch_gmail.rb` – Phase 1: Gmail OAuth2 fetch, email storage as markdown * `summarize.rb` – Phase 2: Claude API summarization with structured output * `build_digest.rb` – Phase 3: weighted ranking + editorial generation + digest output * `digest.rb` – orchestrator calling all three phases in sequence * `prompts/summarize.txt` – structured Claude prompt (summary + score + takeaways) * `prompts/rank.txt` – Claude editorial prompt * Gemfile – single gem dependency (`google-apis-gmail_v1`) * Directory structure: `data/raw/`, `data/summaries/`, `data/digests/` Missing / TBD (requires your input) Item Blocker `credentials.json` Obtain from Google Cloud Console – first next step `token.yaml` Auto-generated on first OAuth run `SENDER_FILTER` in `fetch_gmail.rb` Your actual sender email addresses `SENDER_PRIORITY` in `build_digest.rb` Your priority weighting for those senders `ANTHROPIC_API_KEY` env var Set in shell profile Next Steps (in order) 1. Google Cloud Console: create project, enable Gmail API, download `credentials.json` to `~/Desktop/aggregation/` 2. `export ANTHROPIC_API_KEY=...` in `~/.zshrc` 3. Populate `SENDER_FILTER` and `SENDER_PRIORITY` constants 4. `cd ~/Desktop/aggregation && bundle install` 5. `ruby fetch_gmail.rb` – triggers browser OAuth on first run 6. Verify files appear in `data/raw/`, then run `ruby summarize.rb`, then `ruby build_digest.rb` 7. Full run: `ruby digest.rb`

Working Directory Update

All project files live at `~/bin/attn/version_two/`. References to `~/Desktop/aggregation/` in earlier notes are superseded by this path. The directory already exists; do not recreate or overwrite existing files without explicit instruction.

v1.1 Plan – Claude Code / Cowork Native Version

Context

The Ruby pipeline (v0) is portable and self-contained. v1.1 adds a parallel implementation using Claude Code primitives: a Skill, bundled reference files, an MCP server for Gmail access, and native scheduling. The goal is not feature parity but demonstrating that the same workflow can be expressed in both paradigms – making the project a more complete calling card and a useful reference for open-source audiences choosing an approach.

Key simplification vs v0: **sender_priority** and **recency** are removed as scoring factors. All emails come from the same trusted sender set, all within the past 24 hours – those metrics are constants, not discriminators. The only scoring axis is Claude’s content quality assessment against a plain-language preferences file.

v1.1 is a follow-on, not a parallel build. The SKILL.md workflow, SCORING.md rules, PREFERENCES.md template, and both README files will be extracted from and generated against the v0 Ruby project files – prompts, constants, and structure. No content is duplicated; v1.1 re-expresses what v0 already defines. Nothing in this section gets built until v0 is complete and stable.

New Files – directory structure

```
~/bin/attn/version_two/
  README.md                # NEW: project overview, both approaches explained
  claude-native/           # NEW: Claude Code / Cowork native version
    SKILL.md               # Core workflow skill (fetch -> summarize -> rank -> digest)
    .mcp.json              # Gmail MCP config for Claude Code users
  references/
    SCORING.md             # Content quality rules: what makes a top story
    PREFERENCES.md         # Topic interests, newsletter context, output format
    README.md              # Setup: Cowork path vs Claude Code path, installation
```

Existing Ruby v0 files are unchanged.

SKILL.md

Format: YAML frontmatter + markdown body. Follows the pattern at
~/.claude/skills/code-skillz/SKILL.md.

Frontmatter:

```
---
name: content-digest
description: Daily newsletter aggregation workflow. Fetches emails via Gmail, summarizes
```

each, selects top stories based on content quality, and produces a markdown digest.
Use when running a scheduled or manual digest of newsletter subscriptions.

Body – three-phase workflow Claude executes in sequence:

Phase 1 (Fetch): Use the Gmail tool to search for messages matching `SENDER_FILTER` from the past 24 hours. Store each as a markdown file in `data/raw/` with a header block (sender, subject, date) and plain-text body.

Phase 2 (Summarize): For each file in `data/raw/`, read the content and produce a structured summary. Load `references/SCORING.md` to apply content quality criteria. Output: 2-3 sentence summary, importance rating (1-10 with brief rationale), 3 key takeaways. Store in `data/summaries/`.

Phase 3 (Digest): Read all summaries. Apply comparison rules from `references/SCORING.md` and topic preferences from `references/PREFERENCES.md`. Select top 5 by content quality. Write digest to `data/digests/YYYYMMDD_digest.md` and print to stdout.

Key instruction in `SKILL.md` – reference the bundled files explicitly so Claude loads them at the right phase:

Before scoring any summary, read `references/SCORING.md`. Before selecting top stories, read `references/PREFERENCES.md`.

references/SCORING.md

Replaces the weighted formula from v0. Plain-language rules for content evaluation:

- What signals high importance (novel data, named sources, specific claims, actionability)
- What signals low importance (routine roundups, promotional content, opinion without evidence)
- How to compare two stories of similar quality (specificity, timeliness, uniqueness)
- What a score of 1, 5, and 10 looks like in concrete terms
- Instructions to score comparatively across the day's batch, not in absolute isolation

This is the equivalent of `rank.txt` + weighted formula from v0, unified into one readable reference file.

references/PREFERENCES.md

Ships as a blank template with commented instructions only – section headers and guidance but no example content. Cleaner for open source: no assumptions about the user’s topic interests. Sections:

- Newsletter Sources – list sender names and their domain/topic
- High Priority Topics – topics that should score higher
- Deprioritize – topics or content types to down-rank
- Digest Format – preferred length, tone, output structure
- Standing Rules – freeform instructions Claude should always apply

Primary customization surface for end users of the open-source project.

.mcp.json – Claude Code path

For Claude Code users without Cowork. Configures a Gmail MCP server via stdio. Specific npm package name TBD – several community Gmail MCP servers exist. Credential model will match whichever server is selected.

Cowork path: Gmail is a native Cowork plugin, installed through the Cowork UI. No .mcp.json needed. Scheduling is also native to Cowork (announced Feb 24, 2026). The .mcp.json is only relevant for Claude Code headless/cron users.

Setup paths (claude-native/README.md)

Path A – Claude Cowork: 1. Copy claude-native/ to ~/.claude/skills/content-digest/ 2. Connect Gmail via Cowork’s Gmail plugin 3. Configure scheduling via Cowork’s native scheduler 4. Edit references/PREFERENCES.md

Path B – Claude Code (headless + cron): 1. Copy claude-native/ to ~/.claude/skills/content-digest/ 2. Configure .mcp.json with Gmail MCP server credentials 3. Schedule via cron: 0 7 * * * claude –skill content-digest –headless 4. Edit references/PREFERENCES.md

Project-level README.md

- What the tool does (one paragraph)
 - Approach A – Ruby pipeline (v0): portable, any LLM provider, no Claude Code dependency, swap 5 constants to target any API
 - Approach B – Claude Code / Cowork native (v1.1): no coding required, lower barrier for existing Claude Code / Cowork subscribers, less portable
 - When to choose each; prerequisites per approach
-

Decisions Made (v1.1)

Decision	Choice	Rationale
Scoring model	SCORING.md plain-language rules, no weighted formula	sender_priority and recency are constants for newsletter use case – not discriminators
Skill file location	claude-native/ subdirectory, installable	Keeps both approaches in one repo
Gmail for Cowork	Native plugin, no MCP config	Cowork announced Gmail as built-in integration
Gmail for Claude Code	stdio MCP server via .mcp.json	Standard Claude Code MCP pattern
Scheduling for Cowork	Native Cowork scheduler	Announced Feb 24, 2026
Scheduling for Claude Code	cron + headless mode	Existing Claude Code capability
PREFERENCES.md format	Blank template with commented instructions	No assumptions about user interests; cleaner for open source
Specific Gmail MCP package	TBD	Requires evaluating available community servers

Deferred (v1.1)

Item	Notes
All v1.1 file generation	Blocked on v0 completion; content derived from v0 sources
Gmail MCP server selection	Research specific npm package, document OAuth flow
Plugin/marketplace packaging Cowork scheduling config syntax	Could formalize as installable plugin; v2+ Syntax TBD pending Cowork docs maturing

Verification (v1.1)

1. Install skill: copy claude-native/ to ~/.claude/skills/content-digest/
2. Load skill in Claude Code: confirm SKILL.md is accessible
3. Confirm references/SCORING.md and references/PREFERENCES.md
load when referenced

4. Run Phase 1 manually (Claude Code + Gmail MCP): confirm data/raw/ populates
 5. Run Phase 2: confirm data/summaries/ populates with scored entries
 6. Run Phase 3: confirm digest appears in data/digests/ and stdout
 7. Verify README setup paths are accurate for both Cowork and Claude Code
-